

Searching Algorithms Lab

Data Structures & Algorithms

Lab 11 -- Implementing Search Algorithms on Data Structures

Duration: 1 hour (1 challenge)

Lab Structure

This lab session follows the format:

1. **Challenge Presentation** (5 min)
2. **Your Implementation Time** (25 min)
3. **Pseudocode Review** (5 min)
4. **Solution Walkthrough** (10 min)
5. **Code Explanation** (15 min)

Total: ~60 minutes

Today's Challenge

Challenge 1: Find the Target -- Binary Search

- Implement the **Binary Search** algorithm on a sorted array of integers.
- Track the number of **comparisons** performed during the search.
- Compare its performance against a standard Linear Search for both successful and unsuccessful searches.

Quick Recap: Searching Fundamentals

A **searching algorithm** retrieves information stored within some data structure, or calculates that the information doesn't exist.

Search Categories

Algorithm	Pre-requisite	Mechanism
Linear Search	None (works on unsorted)	Checks every element sequentially from start to end
Binary Search	Must be sorted	Repeatedly divides the search interval in half

Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

Challenge 1

Find the Target -- Binary Search

Challenge 1: Problem Statement

Scenario: You have a sorted array of 15 elements and you need to find the index of specific targets.

```
Array: {2, 5, 8, 12, 16, 23, 38, 56, 72, 91, 105, 120, 134, 150, 180}
```

Tasks:

1. Implement **Binary Search** to find a given target. Print the search space (low and high indices) at each step.
2. Implement a simple **Linear Search** for comparison.
3. Search for the target **23** (exists) using both methods. Output the number of comparisons.
4. Search for the target **85** (does not exist) using both methods. Output comparisons.

Time: 25 minutes

Challenge 1: Think About It First

Binary Search -- "Divide and Conquer"

```
Step 1: Start with the entire array (low = 0, high = n-1).  
Step 2: Find the middle element.  
Step 3: If target == middle, you're done!  
Step 4: If target < middle, narrow search to the left half (high = mid - 1).  
Step 5: If target > middle, narrow search to the right half (low = mid + 1).  
Step 6: Repeat until target is found or low > high (not found).
```

Key Insight: Every single comparison eliminates *half* of the remaining possibilities! This is why it is $O(\log n)$.

Challenge 1: Binary Search Trace

Array: [2, 5, 8, 12, 16, 23, 38, 56, 72, 91, 105, 120, 134, 150, 180]

Target: 23

Step 1: low = 0, high = 14. mid = $(0+14)/2 = 7$. arr[7] = 56.
23 < 56. Target must be in left half. Update high = 6.

Step 2: low = 0, high = 6. mid = $(0+6)/2 = 3$. arr[3] = 12.
23 > 12. Target must be in right half. Update low = 4.

Step 3: low = 4, high = 6. mid = $(4+6)/2 = 5$. arr[5] = 23.
23 == 23. Target found at index 5!

Total comparisons: 3 (Linear search would have taken 6)

Challenge 1: Pseudocode

```
ALGORITHM BinarySearch(arr, n, target)
    low = 0
    high = n - 1
    comparisons = 0

    While low <= high:
        comparisons++
        mid = low + (high - low) / 2

        Print: "Searching between index low and high, mid is mid"

        If arr[mid] == target:
            Return mid, comparisons

        If arr[mid] < target:
            low = mid + 1    // Target is in right half
        Else:
            high = mid - 1   // Target is in left half

    Return -1, comparisons // Target not found
END ALGORITHM
```

Challenge 1: Solution Code -- Linear Search

```
#include <stdio.h>

#define SIZE 15

// Returns index if found, -1 otherwise. Updates comparisons count.
int linearSearch(int arr[], int n, int target, int *comparisons) {
    *comparisons = 0;

    for (int i = 0; i < n; i++) {
        (*comparisons)++;
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}
```

Challenge 1: Solution Code -- Binary Search

```
int binarySearch(int arr[], int n, int target, int *comparisons) {
    int low = 0;
    int high = n - 1;
    *comparisons = 0;

    while (low <= high) {
        (*comparisons)++;
        // Avoids integer overflow for very large arrays
        int mid = low + (high - low) / 2;

        printf("    [Low: %d, High: %d, Mid: %d (val: %d)]\n",
               low, high, mid, arr[mid]);

        if (arr[mid] == target)
            return mid;

        if (arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
```

Challenge 1: Solution Code -- Main

```
int main() {
    int arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91, 105, 120, 134, 150, 180};
    int targets[] = {23, 85}; // 23 exists, 85 does not
    int compLinear, compBinary, result;

    printf("=== Challenge 1: Binary vs Linear Search ===\n\n");

    for(int i=0; i<2; i++) {
        int t = targets[i];
        printf("--- Searching for Target: %d ---\n", t);

        // Linear Search
        result = linearSearch(arr, SIZE, t, &compLinear);
        printf("  Linear Search: Found at idx %2d | Comparisons: %d\n",
               result, compLinear);

        // Binary Search
        printf("  Binary Search steps:\n");
        result = binarySearch(arr, SIZE, t, &compBinary);
        printf("  Binary Search: Found at idx %2d | Comparisons: %d\n\n",
               result, compBinary);
    }
    return 0;
}
```

Challenge 1: Code Explanation

Part 1: Calculating `mid` safely

You might have seen `mid = (low + high) / 2;`

Why did we use `mid = low + (high - low) / 2;` instead?

If `low` and `high` are very large integers (near the maximum value a 32-bit int can hold, ~2.14 billion), adding them together can cause an **integer overflow**.

```
// Example with max 8-bit integer (255)
low = 200, high = 250
mid = (200 + 250) / 2 // 450 overflows 8-bit int! Result is wrong.
mid = 200 + (250 - 200) / 2 // 200 + 50/2 = 200 + 25 = 225. Correct!
```

This is a standard best practice in industry implementations of Binary Search!

Challenge 1: Code Explanation

Part 2: Linear vs Binary Performance

When target exists (e.g., 23):

- Linear Search checks 2, 5, 8, 12, 16, 23 (6 comparisons).
- Binary Search checks 56, 12, 23 (3 comparisons).

When target DOES NOT exist (e.g., 85):

- Linear Search must check *every single element* before giving up (15 comparisons).
- Binary Search quickly narrows the space down to nothing (4 comparisons).

As the array grows, Binary Search's advantage becomes massive. For 1 million elements, worst-case linear takes 1,000,000 comparisons, while binary search takes at most 20! ($2^{20} > 1,000,000$)

Lab Session Complete!

Summary: What You Built

Challenge 1: Binary Search

- Implemented **Binary Search** and saw its divide-and-conquer strategy in action.
- Tracked internal state (`low` , `high` , `mid`) to visualize the search space shrinking.
- Learned the safe way to calculate the midpoint to prevent integer overflow.
- Compared against Linear Search and observed the dramatic difference in the number of comparisons, especially for elements that do not exist in the array.

Key Concepts Reinforced

Linear Search

- **Pros:** Extremely simple to write. Works on any data structure (arrays, linked lists) whether sorted or unsorted.
- **Cons:** Slow ($O(n)$). Unusable for searching large datasets.

Binary Search

- **Pros:** Incredibly fast ($O(\log n)$). Can search massive datasets in just a few dozen comparisons.
- **Cons:** Requires the data to be **sorted** first. Only works on data structures with random access (like Arrays or balanced trees), not standard Linked Lists.

The Sorting Prerequisite

Sorting takes $O(n \log n)$. Binary search takes $O(\log n)$. If you only need to search *once*, just use linear search ($O(n)$). If you need to search *many times*, sort it once, then use binary search repeatedly!

Take-Home Challenge (Optional)

Extend your implementation with these exercises:

1. **Recursive Binary Search:** Implement the binary search algorithm recursively instead of using a `while` loop. The function signature should look like: `int recursiveBinary(int arr[], int low, int high, int target)`.
2. **First/Last Occurrence:** If the array contains duplicates (e.g., `{2, 4, 4, 4, 8, 9}`), standard binary search just returns *one* of the indices. Modify your code to find the index of the **first occurrence** of a target.
3. **String Search:** Modify your code to search an array of alphabetically sorted strings instead of integers. (Hint: Use `strcmp()` from `<string.h>`).